

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 2
по дисциплине «Методы машинного обучения»
на тему
«Знакомство с инструментами сбора и подготовки данных»
Вариант 17

Выполнил: студент гр. 351002

Проверил:

Лашенко Е.Н.

Лебедевич А.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью работы является освоение загрузки табличных данных, поиска и устранения пропусков и аномалий, построения новых признаков, кодирования категорий, а также стандартизации и нормализации. Индивидуальное задание варианта 17 выполнено на наборе Student Performance Factors.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Таблицу наблюдений хранят в объекте DataFrame. У столбца один тип: целое число или строка. Пропуск – это отсутствующее значение. Пока его не заполнили и строку не удалили, обычные сводки по столбцу этот пропуск не учитывают.

Долю пропусков получают делением числа пустых ячеек столбца на число строк. Если доля мала и пустые ячейки разбросаны, строку целиком не удаляют: вместе с пропуском пропадает заполненная часть наблюдения. В категориальном столбце пропуск заменяют модой, то есть самым частым уровнем. Мода не переносит центр категории, а только усиливает уже преобладающий уровень.

Числовой столбец описывают средним, стандартным отклонением, минимумом и максимумом. Гистограмма показывает, где лежит основная масса значений и есть ли длинный хвост. Диаграмма размаха отмечает квартили и отдельно выносит точки за границами «усов». Тепловая карта пропусков ставит отметку в пустой ячейке, поэтому видно, собраны пропуски блоком или рассеяны по строкам.

Линейную связь двух числовых признаков измеряют коэффициентом корреляции. Значение около нуля означает, что линейной связи нет. Значение около единицы означает, что признаки растут согласованно. Набор коэффициентов выводят той же тепловой картой.

Выбросом по правилу Тьюки считают наблюдение вне границ межквартильного размаха. Размах и границы определяют по формулам:

$$IQR = Q_3 - Q_1, \quad (2.1)$$

$$L = Q_1 - 1,5 \cdot IQR, \quad (2.2)$$

$$U = Q_3 + 1,5 \cdot IQR, \quad (2.3)$$

где IQR – межквартильный размах;

Q_1 – первый квартиль;

Q_3 – третий квартиль;

L – нижняя граница;

U – верхняя граница.

Коэффициент 1,5 оставляет центральную половину выборки и обычные хвосты, а редкие края отсекает. У узкого распределения граница задевает и допустимые редкие значения. У скошенного распределения одна граница

уходит за предметную область, и отсекается только противоположный хвост. Ошибку шкалы, например балл выше допустимого максимума, сначала ограничивают этим максимумом.

Новый признак получают из уже имеющихся, если сочетание несёт отдельный смысл: произведение двух величин, их разность или сумма. Категорию с упорядоченными уровнями заменяют целым кодом и сохраняют порядок. Два уровня без порядка заменяют нулём и единицей. Несколько неупорядоченных уровней раскладывают в индикаторы: каждый уровень, кроме опорного, получает свой столбец. Опорный уровень соответствует случаю, когда все индикаторы равны нулю, поэтому столбцы не повторяют друг друга.

После кодирования столбцы лежат в разных шкалах. Метод, который сравнивает строки по расстоянию, тогда считает крупный по числам признак более важным. Стандартизация снимает масштаб:

$$z = \frac{x - \mu}{\sigma}, \quad (2.4)$$

где x – исходное значение;

μ – среднее столбца;

σ – стандартное отклонение столбца.

Нормализация укладывает столбец в отрезок от нуля до единицы:

$$y = \frac{x - x_0}{x_1 - x_0}, \quad (2.5)$$

где x_0 – минимум столбца;

x_1 – максимум столбца.

Оба преобразования сохраняют порядок значений внутри столбца. Меняются только положение нуля и цена деления.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Задание выполнено по варианту 17. Набор данных выбран самостоятельно: Student Performance Factors, файл StudentPerformanceFactors.csv.

1 Импортировать набор данных.

2 Проверить пропуски и аномальные значения статистикой и графиками: распределения, диаграммы размаха, тепловые карты. Заполнить пропуски, устранить выбросы и графически убедиться, что пропуски и аномалии отсутствуют.

3 Сгенерировать не менее двух новых признаков. Закодировать категориальные признаки: бинаризация, порядковое кодирование и one-hot.

4 Выполнить стандартизацию и нормализацию подготовленной матрицы.

4 РЕШЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Решение записано на языке Python с библиотеками pandas, NumPy, matplotlib, seaborn и scikit-learn. Ниже приведён код в том виде, в каком он выполнен в ноутбуке, и результаты запуска.

4.1 Импорт набора

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    OneHotEncoder,
    LabelBinarizer,
    OrdinalEncoder,
)

sns.set_theme(style="whitegrid", palette="muted")
pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 120)

print("Библиотеки загружены.")

DATA_PATH = "StudentPerformanceFactors/StudentPerformanceFactors.csv"

df = pd.read_csv(DATA_PATH)
print(f"Размерность: {df.shape[0]} строк × {df.shape[1]} столбцов")
df.head()

print("Типы данных:")
print(df.dtypes)
print("\nОбщая информация:")
df.info()

df.describe(include="all").T
```

После загрузки таблица содержит 6607 строк и 20 столбцов. Семь столбцов имеют тип int64, тринадцать – тип str. Пропуски уже видны в сводке: Teacher_Quality заполнен в 6529 строках, Parental_Education_Level – в 6517, Distance_from_Home – в 6540. Максимум Exam_Score равен 101 и выходит за шкалу балла.

4.2 Пропуски

```
missing = df.isna().sum()
```

```

missing_pct = (missing / len(df) * 100).round(2)
missing_table = pd.DataFrame({
    "Пропуски": missing,
    "Доля, %": missing_pct,
})
missing_table = missing_table[missing_table["Пропуски"] >
0].sort_values("Пропуски", ascending=False)

print("Столбцы с пропусками:")
display(missing_table)
print(f"\nВсего пропущенных значений: {int(df.isna().sum().sum())}")
print(f"Строк с хотя бы одним пропуском: {df.isna().any(axis=1).sum()}")

# Тепловая карта пропусков (фрагмент для наглядности)
fig, ax = plt.subplots(figsize=(12, 5))
sns.heatmap(df.isna(), cbar=True, yticklabels=False, cmap="YlOrRd", ax=ax)
ax.set_title("Тепловая карта пропусков (жёлтый/красный = NaN)")
plt.tight_layout()
plt.show()

```

Пустые ячейки есть в трёх столбцах: `Parental_Education_Level` – 90 (1,36 %), `Teacher_Quality` – 78 (1,18 %), `Distance_from_Home` – 67 (1,01 %). Всего 235 пропусков в 229 строках. Расположение пропусков показано на рисунке 4.1.

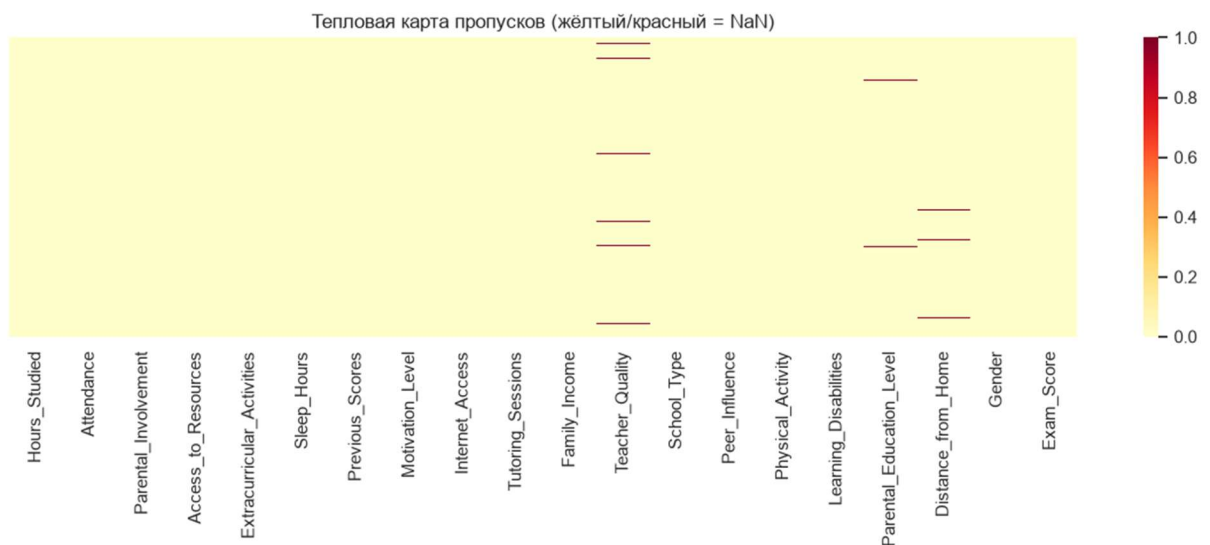


Рисунок 4.1 – Тепловая карта пропусков в исходной таблице

4.3 Распределения, диаграммы размаха и корреляция

```

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in df.columns if c not in num_cols]

print("Числовые признаки:", num_cols)
print("Категориальные признаки:", cat_cols)

df[num_cols].describe().T

```

```

# Графики распределения числовых признаков
n = len(num_cols)
ncols = 3
nrows = int(np.ceil(n / ncols))

fig, axes = plt.subplots(nrows, ncols, figsize=(14, 3.2 * nrows))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.histplot(df[col], kde=True, ax=axes[i], bins=25)
    axes[i].set_title(f"Распределение: {col}")

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()

# BoxPlot для выявления выбросов
fig, axes = plt.subplots(nrows, ncols, figsize=(14, 3.2 * nrows))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.boxplot(x=df[col], ax=axes[i], color="skyblue")
    axes[i].set_title(f"BoxPlot: {col}")

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()

# Корреляционная тепловая карта
fig, ax = plt.subplots(figsize=(9, 7))
corr = df[num_cols].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax)
ax.set_title("Корреляционная матрица числовых признаков")
plt.tight_layout()
plt.show()

```

К числовым отнесены семь признаков, к категориальным – тринадцать. Распределения приведены на рисунке 4.2: балл экзамена сосредоточен около 67, число занятий с репетитором скошено вправо. Диаграммы размаха на рисунке 4.3 выделяют хвосты у Hours_Studied, Tutoring_Sessions и Exam_Score. Корреляционная матрица приведена на рисунке 4.4. С баллом экзамена сильнее всего связаны посещаемость (0,581) и часы учёбы (0,445). Остальные числовые признаки между собой почти не связаны.

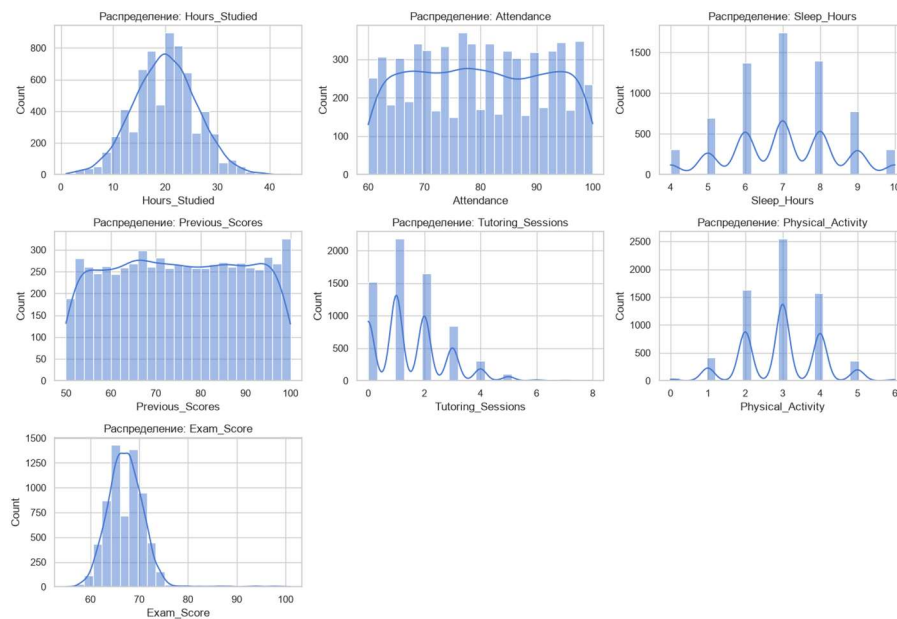


Рисунок 4.2 – Распределения числовых признаков

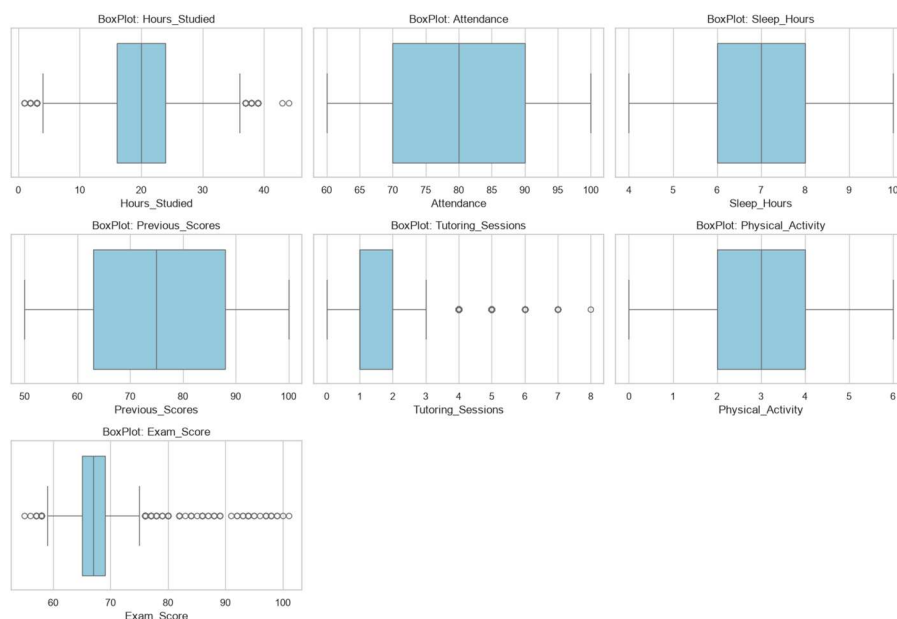


Рисунок 4.3 – Диаграммы размаха числовых признаков

Часы учёбы дают редкие значения и ниже 4, и выше 36. Занятия с репетитором имеют правый хвост до 8 при типичных значениях от 0 до 3. Балл экзамена сжат около 67, поэтому за границами «усов» оказываются и низкие, и высокие оценки. Посещаемость, сон, предыдущие оценки и физическая активность таких точек не дают. Числовые связи этих признаков с баллом экзамена собраны на рисунке 4.4.

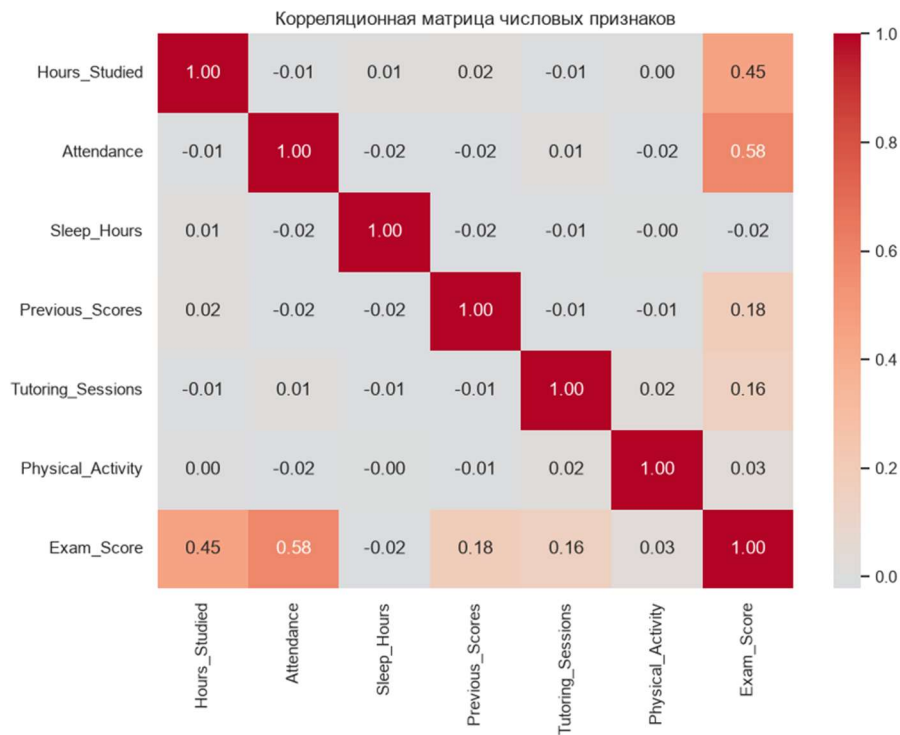


Рисунок 4.4 – Корреляционная матрица числовых признаков

4.4 Границы выбросов, заполнение пропусков и очистка

Границы выбросов вычислены по формулам (2.1)–(2.3).

```
def iqr_bounds(series: pd.Series, k: float = 1.5):
    """Границы выбросов по правилу межквартильного размаха."""
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - k * iqr
    upper = q3 + k * iqr
    return lower, upper

outlier_summary = []
for col in num_cols:
    lower, upper = iqr_bounds(df[col])
    mask = (df[col] < lower) | (df[col] > upper)
    outlier_summary.append({
        "Признак": col,
        "Нижняя граница": round(lower, 2),
        "Верхняя граница": round(upper, 2),
        "Выбросов (IQR)": int(mask.sum()),
        "min": df[col].min(),
        "max": df[col].max(),
    })

outlier_df = pd.DataFrame(outlier_summary)
display(outlier_df)
```

```
# Предметная аномалия: балл экзамена не может быть > 100
invalid_exam = df["Exam_Score"] > 100
print(f"\nExam_Score > 100: {invalid_exam.sum()} записей")
if invalid_exam.any():
    display(df.loc[invalid_exam, ["Hours_Studied", "Attendance",
    "Previous_Scores", "Exam_Score"]])
```

По Hours_Studied найдено 43 выброса (границы от 4 до 36): 21 значение меньше 4 и 22 значения больше 36. По Tutoring_Sessions – 430 выбросов, все это уровни от 4 до 8 занятий. По Exam_Score – 104 выброса (границы от 59 до 75): 28 значений ниже 59 и 76 выше 75. У Attendance, Sleep_Hours, Previous_Scores и Physical_Activity выбросов нет. Одно наблюдение имеет Exam_Score, равный 101.

Пропуски заполнены модой, балл ограничен сверху сотней, строки с выбросами по трём признакам удалены.

```
df_clean = df.copy()

# --- Пропуски в категориальных признаках: мода ---
cat_with_na = ["Teacher_Quality", "Parental_Education_Level",
"Distance_from_Home"]
for col in cat_with_na:
    mode_val = df_clean[col].mode(dropna=True)[0]
    n_before = df_clean[col].isna().sum()
    df_clean[col] = df_clean[col].fillna(mode_val)
    print(f"{col}: заполнено {n_before} пропусков значением '{mode_val}'")

print(f"\nПропусков после заполнения: {int(df_clean.isna().sum().sum())}")

# --- Выбросы ---
# 1) Очевидная ошибка: Exam_Score > 100 → ограничиваем сверху (winsorize /
clip)
n_clip_exam = (df_clean["Exam_Score"] > 100).sum()
df_clean["Exam_Score"] = df_clean["Exam_Score"].clip(upper=100)
print(f"Exam_Score: обрезано сверху {n_clip_exam} значений (> 100 → 100)")

# 2) Остальные числовые признаки: удаление строк с выбросами по IQR
# (кроме Attendance/Sleep_Hours, где диапазон естественный и узкий –
оставляем IQR-фильтр
# для признаков с заметными «хвостами»: Hours_Studied, Tutoring_Sessions,
Exam_Score)
cols_for_iqr_removal = ["Hours_Studied", "Tutoring_Sessions", "Exam_Score"]

mask_keep = pd.Series(True, index=df_clean.index)
for col in cols_for_iqr_removal:
    lower, upper = iqr_bounds(df_clean[col])
    col_mask = (df_clean[col] >= lower) & (df_clean[col] <= upper)
    removed = (~col_mask).sum()
    print(f"{col}: удаляется {removed} выбросов (границы [{lower:.2f};
{upper:.2f}])")
    mask_keep &= col_mask

rows_before = len(df_clean)
df_clean = df_clean.loc[mask_keep].reset_index(drop=True)
rows_after = len(df_clean)
```

```
print(f"\nСтрок до очистки: {rows_before}")
print(f"Строк после очистки: {rows_after}")
print(f"Удалено строк: {rows_before - rows_after}")
```

Модой заполнены Teacher_Quality (Medium, 78 пропусков), Parental_Education_Level (High School, 90) и Distance_from_Home (Near, 67). Значение 101 заменено на 100 и всё равно удалено фильтром балла, так как 100 больше верхней границы 75. Удалено 553 строки. В выборке осталось 6054 строки.

4.5 Проверка очистки

```
print("Пропуски после очистки:")
print(df_clean.isna().sum())
print(f"\nСумма пропусков: {int(df_clean.isna().sum().sum())}")
print(f"Exam_Score > 100: {(df_clean['Exam_Score'] > 100).sum()}")

# Тепловая карта пропусков после очистки (должна быть «пустой»)
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

sns.heatmap(df.isna(), cbar=False, yticklabels=False, cmap="YlOrRd",
ax=axes[0])
axes[0].set_title("До очистки")

sns.heatmap(df_clean.isna(), cbar=False, yticklabels=False, cmap="YlOrRd",
ax=axes[1])
axes[1].set_title("После очистки (пропусков нет)")

plt.tight_layout()
plt.show()

# BoxPlot до / после для ключевых признаков с выбросами
compare_cols = ["Hours_Studied", "Tutoring_Sessions", "Exam_Score"]

fig, axes = plt.subplots(2, len(compare_cols), figsize=(12, 6))
for i, col in enumerate(compare_cols):
    sns.boxplot(x=df[col], ax=axes[0, i], color="salmon")
    axes[0, i].set_title(f"До: {col}")
    sns.boxplot(x=df_clean[col], ax=axes[1, i], color="lightgreen")
    axes[1, i].set_title(f"После: {col}")

plt.suptitle("Сравнение BoxPlot до и после устранения выбросов", y=1.02)
plt.tight_layout()
plt.show()

# Контроль IQR после очистки
check_rows = []
for col in compare_cols:
    lower, upper = iqr_bounds(df_clean[col])
    n_out = ((df_clean[col] < lower) | (df_clean[col] > upper)).sum()
    check_rows.append({"Признак": col, "Выбросов IQR после": int(n_out)})

display(pd.DataFrame(check_rows))
print("Графически и по IQR аномальные значения в целевых признаках устранены / сведены к минимуму.")
```

Сумма пропусков после очистки равна нулю, баллов выше 100 нет. Повторный расчёт границ для Hours_Studied, Tutoring_Sessions и Exam_Score даёт ноль выбросов. Сравнение тепловых карт до и после очистки показано на рисунке 4.5. Правая карта пустая.

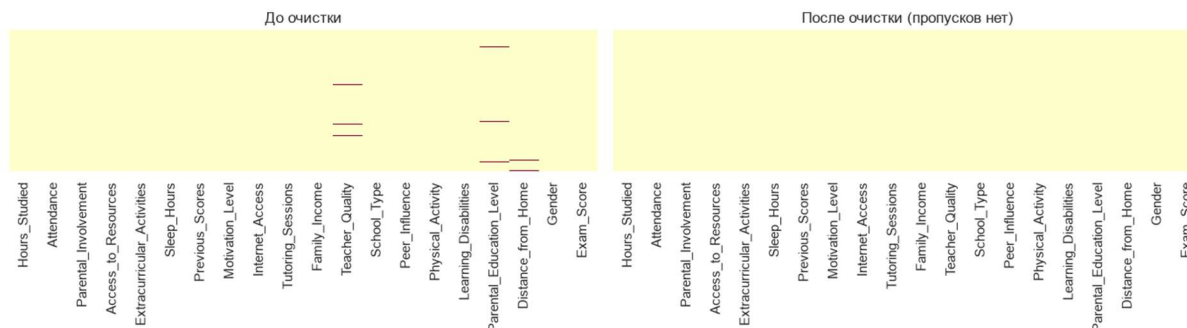


Рисунок 4.5 – Пропуски до и после очистки

Диаграммы размаха тех же трёх признаков до и после удаления строк приведены на рисунке 4.6.

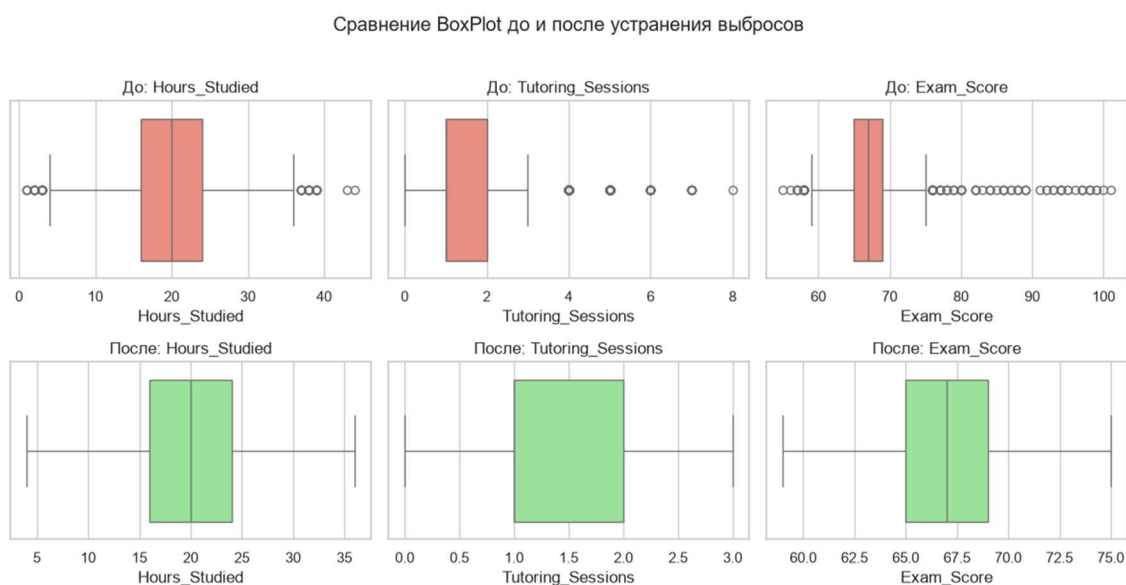


Рисунок 4.6 – Диаграммы размаха до и после удаления выбросов

4.6 Новые признаки

Построены четыре признака: интенсивность учёбы, прирост балла, баланс сна и суммарная учебная нагрузка.

```
df_feat = df_clean.copy()
```

```

# 1) Интенсивность учёбы: часы учёбы с учётом посещаемости
df_feat["Study_Intensity"] = df_feat["Hours_Studied"] *
(df_feat["Attendance"] / 100)

# 2) Прирост относительно прошлых оценок
df_feat["Score_Gain"] = df_feat["Exam_Score"] - df_feat["Previous_Scores"]

# 3) Дополнительно: баланс сна (насколько сон близок к «норме» 7-8 ч)
df_feat["Sleep_Balance"] = 1 - (df_feat["Sleep_Hours"] - 7.5).abs() / 7.5

# 4) Учебная нагрузка = учёба + репетиторство
df_feat["Total_Learning_Load"] = df_feat["Hours_Studied"] +
df_feat["Tutoring_Sessions"]

new_features = ["Study_Intensity", "Score_Gain", "Sleep_Balance",
"Total_Learning_Load"]
print("Добавлены признаки:", new_features)
df_feat[new_features].describe().T

fig, axes = plt.subplots(1, 4, figsize=(14, 3.5))
for ax, col in zip(axes, new_features):
    sns.histplot(df_feat[col], kde=True, ax=ax, bins=25)
    ax.set_title(col)
plt.tight_layout()
plt.show()

```

На 6054 строках среднее Study_Intensity равно 15,99, среднее Score_Gain равно -8,13, среднее Sleep_Balance равно 0,83, среднее Total_Learning_Load равно 21,31. Распределения показаны на рисунке 4.7.

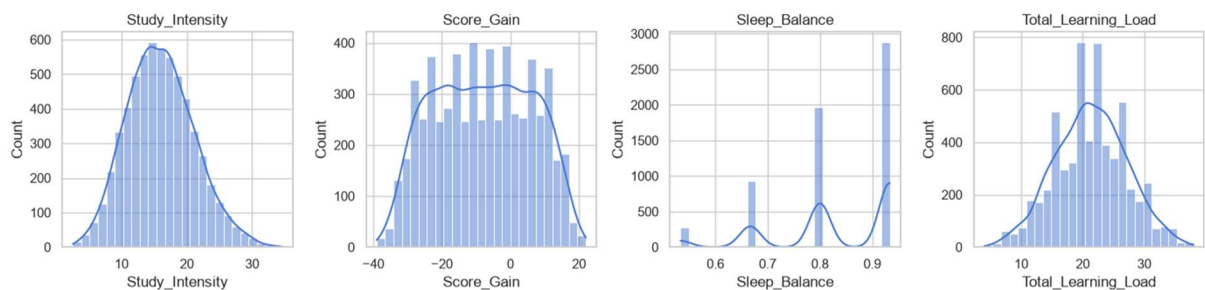


Рисунок 4.7 – Распределения новых признаков

4.7 Кодирование категорий

Двоичные признаки переведены в 0 и 1, упорядоченные уровни – в коды 0, 1 и 2, признаки без порядка разложены one-hot с отбрасыванием опорного уровня.

```

df_enc = df_feat.copy()

# Бинарные Yes/No → 1/0
binary_yes_no = [
    "Extracurricular_Activities",

```

```

        "Internet_Access",
        "Learning_Disabilities",
    ]
    for col in binary_yes_no:
        df_enc[col + "_bin"] = (df_enc[col] == "Yes").astype(int)

    # Gender: Male=1, Female=0
    df_enc["Gender_bin"] = (df_enc["Gender"] == "Male").astype(int)

    # School_Type через LabelBinarizer (Private=1 / Public=0)
    lb = LabelBinarizer()
    df_enc["School_Type_bin"] = lb.fit_transform(df_enc["School_Type"]).ravel()

    bin_cols = [c + "_bin" for c in binary_yes_no] + ["Gender_bin",
    "School_Type_bin"]
    print("Бинаризованные признаки:")
    display(df_enc[bin_cols].head())
    print(df_enc[bin_cols].mean().rename("доля единиц"))

    # Уровни Low < Medium < High
    level_map = {"Low": 0, "Medium": 1, "High": 2}
    ordinal_level_cols = [
        "Parental_Involvement",
        "Access_to_Resources",
        "Motivation_Level",
        "Family_Income",
        "Teacher_Quality",
    ]

    for col in ordinal_level_cols:
        df_enc[col + "_ord"] = df_enc[col].map(level_map)

    # Distance: Near < Moderate < Far
    df_enc["Distance_from_Home_ord"] = df_enc["Distance_from_Home"].map(
        {"Near": 0, "Moderate": 1, "Far": 2}
    )

    # Parental education: High School < College < Postgraduate
    df_enc["Parental_Education_Level_ord"] =
    df_enc["Parental_Education_Level"].map(
        {"High School": 0, "College": 1, "Postgraduate": 2}
    )

    ord_cols = [c + "_ord" for c in ordinal_level_cols] + [
        "Distance_from_Home_ord",
        "Parental_Education_Level_ord",
    ]
    display(df_enc[ord_cols].head())

    # ОНЭ для номинальных признаков без естественного порядка
    ohe_source_cols = ["Peer_Influence", "School_Type", "Gender"]

    ohe = OneHotEncoder(sparse_output=False, drop="first") # drop='first' -
    избегаем дамми-ловушки
    ohe_matrix = ohe.fit_transform(df_enc[ohe_source_cols])
    ohe_names = ohe.get_feature_names_out(ohe_source_cols)
    df_ohe = pd.DataFrame(ohe_matrix, columns=ohe_names, index=df_enc.index)

    print("ОНЭ-столбцы:", list(ohe_names))
    display(df_ohe.head())

    # Объединяем с основным набором

```

```

df_enc = pd.concat([df_enc, df_ohe], axis=1)
print(f"Размерность после ONE: {df_enc.shape}")

# Берём исходные числовые + новые + закодированные
base_num = [
    "Hours_Studied", "Attendance", "Sleep_Hours", "Previous_Scores",
    "Tutoring_Sessions", "Physical_Activity", "Exam_Score",
]

feature_cols = base_num + new_features + bin_cols + ord_cols +
list(ohe_names)
X = df_enc[feature_cols].copy()

print(f"Число признаков для масштабирования: {X.shape[1]}")
print(X.columns.tolist())
X.head()

```

Доля единиц после бинаризации: внеучебная деятельность 0,595, доступ в интернет 0,924, трудности в обучении 0,103, мужской пол 0,577, тип школы Public 0,696. One-hot добавил столбцы Peer_Influence_Neutral, Peer_Influence_Positive, School_Type_Public и Gender_Male. Опорные уровни – Negative, Private и Female. Полная таблица после кодирования имеет размер 6054×40. В матрицу для масштабирования вошли 27 числовых столбцов. Пол и тип школы представлены в ней дважды, потому что к ним применены и бинаризация, и one-hot.

4.8 Стандартизация и нормализация

Стандартизация выполнена по формуле (2.4), нормализация – по формуле (2.5).

```

scaler_std = StandardScaler()
X_standardized = pd.DataFrame(
    scaler_std.fit_transform(X),
    columns=X.columns,
    index=X.index,
)

scaler_mm = MinMaxScaler()
X_normalized = pd.DataFrame(
    scaler_mm.fit_transform(X),
    columns=X.columns,
    index=X.index,
)

print("Стандартизация – mean / std (фрагмент):")
display(pd.DataFrame({
    "mean": X_standardized.mean().round(6),
    "std": X_standardized.std(ddof=0).round(6),
}).head(10))

print("\nНормализация – min / max (фрагмент):")
display(pd.DataFrame({
    "min": X_normalized.min().round(6),

```

```

    "max": X_normalized.max().round(6),
  }).head(10))

# Визуальное сравнение исходных / стандартизированных / нормализованных
значений
demo_cols = ["Hours_Studied", "Attendance", "Previous_Scores", "Exam_Score",
"Study_Intensity"]

fig, axes = plt.subplots(3, len(demo_cols), figsize=(14, 8))

for i, col in enumerate(demo_cols):
    sns.histplot(X[col], ax=axes[0, i], bins=20, color="steelblue", kde=True)
    axes[0, i].set_title(f"Исходный\n{col}")

    sns.histplot(X_standardized[col], ax=axes[1, i], bins=20,
color="darkorange", kde=True)
    axes[1, i].set_title("StandardScaler")

    sns.histplot(X_normalized[col], ax=axes[2, i], bins=20, color="seagreen",
kde=True)
    axes[2, i].set_title("MinMaxScaler")

plt.suptitle("Сравнение распределений до и после масштабирования", y=1.01)
plt.tight_layout()
plt.show()

# Сохранение подготовленных данных
X_standardized.to_csv("StudentPerformanceFactors/prepared_standardized.csv",
index=False)
X_normalized.to_csv("StudentPerformanceFactors/prepared_normalized.csv",
index=False)
df_clean.to_csv("StudentPerformanceFactors/cleaned_raw.csv", index=False)

print("Сохранено:")
print(" - StudentPerformanceFactors/cleaned_raw.csv")
print(" - StudentPerformanceFactors/prepared_standardized.csv")
print(" - StudentPerformanceFactors/prepared_normalized.csv")

```

После стандартизации среднее каждого из 27 столбцов равно нулю, стандартное отклонение равно единице. После нормализации минимум равен нулю, максимум равен единице. Порядок значений внутри столбца сохраняется: меняются только положение нуля и цена деления. На рисунке 4.8 это показано для Hours_Studied, Attendance, Previous_Scores, Exam_Score и Study_Intensity. Верхний ряд – исходные шкалы, средний ряд – результат формулы (2.4), нижний ряд – результат формулы (2.5). Очищенная таблица записана в cleaned_raw.csv. Стандартизованная матрица записана в prepared_standardized.csv. Нормализованная матрица записана в prepared_normalized.csv.

Сравнение распределений до и после масштабирования

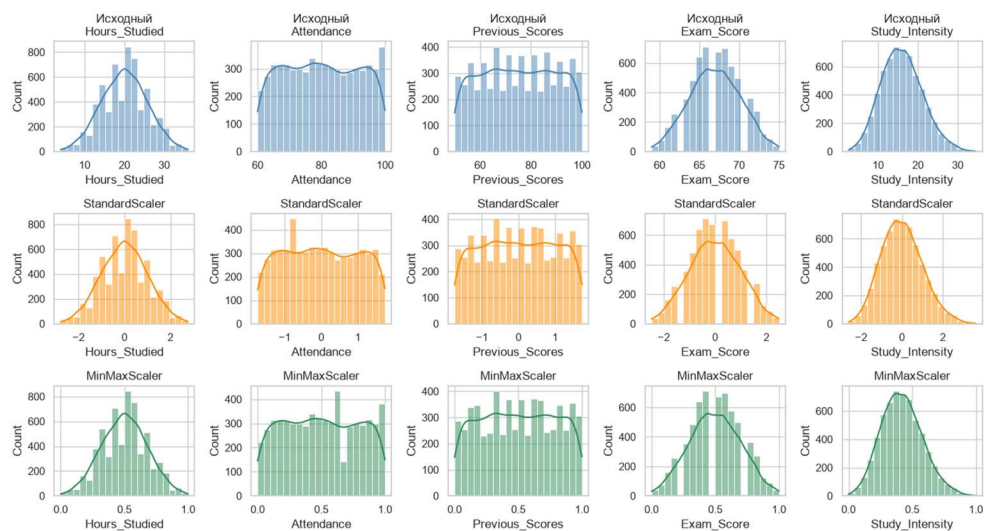


Рисунок 4.8 – Распределения до и после масштабирования

5 ВЫВОДЫ

По варианту 17 подготовлен набор Student Performance Factors: 6607 наблюдений и 20 признаков.

Пропуски в трёх категориальных столбцах заполнены модой. По правилу межквартильного размаха удалены хвосты часов учёбы, занятий с репетитором и балла экзамена, значение 101 ограничено сотней. Осталось 6054 строки. Графики после очистки пропусков не показывают, повторный поиск выбросов по тем же трём признакам даёт ноль.

Построены четыре новых признака. Категории закодированы бинаризацией, порядковыми кодами и one-hot. Матрица для масштабирования содержит 27 столбцов.

Стандартизация привела столбцы к нулевому среднему и единичному отклонению, нормализация – к отрезку от 0 до 1. Обе матрицы и очищенная таблица сохранены. Поставленная цель достигнута.